Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №5
на тему

ХЕШ-ФУНКЦИИ

БГУИР КП 6-05-0612-02 015 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф.Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 8
Приложение А (обязательное) Листинг программного кода 9

ВВЕДЕНИЕ

В данной лабораторной работе рассматриваются криптографические хеш-функции и их применение для контроля целостности сообщений. Хеш-функцией называется необратимое преобразование, которое двоичному сообщению произвольной длины ставит в соответствие двоичный блок фиксированной длины – хеш-значение. Значения на выходе хеш-функции распределены равномерно по всему диапазону, а изменение хотя бы одного бита входного сообщения приводит к непредсказуемому изменению результата: в идеальном случае инвертируется примерно половина бит хеш-значения.

Если для хеш-функции невозможно подобрать другое сообщение с тем же хеш-значением и невозможно найти два различных сообщения с одинаковым хеш-значением, то хеш-значение становится уникальной характеристикой документа. На этом основан контроль целостности: для исходного файла вычисляется и сохраняется контрольная сумма, а при последующей проверке хеш-значение вычисляется повторно и сравнивается с сохранённым. Любое изменение содержимого файла приводит к несовпадению значений.

В работе реализованы две хеш-функции, построенные на идее функции сжатия. Сообщение разбивается на блоки по 512 бит, и хеш-значение каждого блока зависит от самого блока и результата обработки предыдущих блоков:

$h_i = f(M_i, h_{i-1})$.

Первая из них – хеш-функция ГОСТ Р 34.11-2012 «Стрибог», государственный стандарт Российской Федерации, принятый также в качестве межгосударственного стандарта ГОСТ 34.11-2018. Она формирует хеш-значение длиной 256 или 512 бит. Вторая – алгоритм SHA-1, предложенный институтом NIST в качестве стандарта хеширования в гражданской криптографии и формирующий дайджест длиной 160 бит.

Целью лабораторной работы является разработка программного средства на языке C++, выполняющего контроль целостности сообщений с помощью вычисления хеш-функций ГОСТ 34.11 и SHA-1, реализованных самостоятельно, без использования готовых библиотек и функций.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для контроля целостности файлов. Обе хеш-функции реализованы самостоятельно, без использования сторонних библиотек. Программа поддерживает вычисление хеш-значений файла или строки, формирование файла контрольных сумм, проверку целостности файла по сохранённым контрольным суммам, а также проверку реализации на контрольных примерах из стандартов.

Программа разделена на несколько модулей. Модуль streebog реализует хеш-функцию ГОСТ Р 34.11-2012, модуль sha1 – алгоритм SHA-1, модуль integrity – формирование и проверку файла контрольных сумм, а главный модуль main.cpp отвечает за разбор аргументов командной строки и вывод результатов.

В модуле streebog все промежуточные значения являются 512-битными векторами и хранятся как массивы из 64 байт, причём нулевой байт массива является младшим. В алгоритме используются четыре преобразования. X-преобразование выполняет побитовое сложение по модулю 2 двух векторов. S-преобразование заменяет каждый байт вектора по таблице нелинейной биекции π. P-преобразование переставляет байты по правилу

$$\tau(i) = 8 \cdot (i \bmod 8) + \left\lfloor \frac{i}{8} \right\rfloor, \quad i = 0, \ldots, 63,$$

что соответствует транспонированию матрицы байтов размером 8×8. L-преобразование разбивает вектор на восемь 64-битных слов и умножает каждое слово на двоичную матрицу A размером 64×64: если бит слова с номером $63 - i$ равен единице, к результату прибавляется по модулю 2 строка A_i. Последовательное применение трёх последних преобразований обозначается как LPS.

Основным элементом хеш-функции является функция сжатия, вычисляемая по формуле

$$g_N(h, m) = E(\mathrm{LPS}(h \oplus N), m) \oplus h \oplus m,$$

где h – текущее хеш-значение, m – очередной блок сообщения, N – число уже обработанных бит. Функция $E(K, m)$ выполняет 12 раундов. Сначала вычисляется $\mathrm{state} = K \oplus m$, после чего в каждом раунде $i = 1, \ldots, 12$ выполняются действия

$$\mathrm{state} = \mathrm{LPS}(\mathrm{state}), \quad K = \mathrm{LPS}(K \oplus C_i), \quad \mathrm{state} = \mathrm{state} \oplus K,$$

где $C_1, \ldots, C_{12}$ – итерационные константы стандарта. Вычисление раундового ключа $K = \mathrm{LPS}(K \oplus C_i)$ соответствует функции KeySchedule.

Перед обработкой сообщения переменным присваиваются начальные значения: $N = 0$, $\Sigma = 0$, а вектор h равен нулевому вектору для хеш-функции длиной 512 бит и вектору из байтов 0x01 для хеш-функции длиной 256 бит. Пока длина необработанной части сообщения не меньше 512 бит, от неё отделяется блок m и выполняются вычисления

$$h = g_N(h, m), \quad N = (N + 512) \bmod 2^{512}, \quad \Sigma = (\Sigma + m) \bmod 2^{512}.$$

Оставшаяся часть сообщения M длиной менее 512 бит дополняется по правилу $m = 0^{511 - |M|} \parallel 1 \parallel M$, после чего вычисляется $h = g_N(h, m)$, к N при-

бавляется длина $|M|$, к Σ – блок m. Завершают вычисление два преобразования с нулевым значением N:

$h = g_0(h, N), \quad h = g_0(h, \Sigma).$

Для длины выхода 512 бит результатом является вектор h, для длины 256 бит – его старшие 256 бит. В стандарте сообщение и хеш-значение записываются числом, у которого старший байт стоит слева, а обработка начинается с последних 512 бит. Поскольку в программе байты хранятся в порядке возрастания старшинства, блоки берутся последовательно с начала файла, что позволяет обрабатывать данные потоком. Хеш-значение выводится как последовательность байтов, а при сверке с контрольными примерами оно переводится в запись стандарта.

Модуль sha1 реализует алгоритм SHA-1. Сообщение дополняется единичным битом и нулями так, чтобы его длина была сравнима с 448 по модулю 512, после чего добавляется 64-битная длина исходного сообщения. Буфер из пяти 32-битных регистров инициализируется значениями

$A = 67452301, \quad B = \text{EFCDAB89}, \quad C = \text{98BADCFE},$

$D = 10325476, \quad E = \text{C3D2E1F0}.$

Каждый 512-битный блок обрабатывается за 80 циклов. Первые 16 слов W_t берутся непосредственно из блока, остальные вычисляются по формуле

$W_t = \text{CLS}_1(W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3}), \quad 16 \le t \le 79.$

В методических указаниях эта формула приведена без циклического сдвига. Такой вариант соответствует первоначальной версии алгоритма SHA-0, поэтому в программе используется формула из стандарта FIPS 180-4, и только с ней результаты совпадают с контрольными примерами. В каждом цикле содержимое буфера изменяется по правилу

$A, B, C, D, E \leftarrow (\text{CLS}_5(A) + f_t(B, C, D) + E + W_t + K_t), A, \text{CLS}_{30}(B), C, D,$

где сложение выполняется по модулю 2^{32}, а элементарная функция f_t и константа K_t зависят от номера цикла. Для $0 \le t \le 19$ функция является условной, для $20 \le t \le 39$ и $60 \le t \le 79$ она вычисляет бит чётности, а для $40 \le t \le 59$ истинна, если истинны два или три аргумента:

$f_t = (B \wedge C) \vee (\neg B \wedge D), \quad K_t = \text{5A827999}, \quad 0 \le t \le 19;$

$f_t = B \oplus C \oplus D, \quad K_t = \text{6ED9EBA1}, \quad 20 \le t \le 39;$

$f_t = (B \wedge C) \vee (B \wedge D) \vee (C \wedge D), \quad K_t = \text{8F1BBCDC}, \quad 40 \le t \le 59;$

$f_t = B \oplus C \oplus D, \quad K_t = \text{CA62C1D6}, \quad 60 \le t \le 79.$

После 80 циклов полученные значения регистров прибавляются по модулю 2^{32} к значениям буфера до обработки блока. После обработки последнего блока содержимое буфера образует 160-битный дайджест сообщения.

Модуль integrity выполняет контроль целостности. При формировании контрольных сумм для файла вычисляются три хеш-значения: ГОСТ 34.11-2012 длиной 256 и 512 бит и SHA-1. Они записываются в текстовый файл вместе с именем и размером исходного файла. При проверке файл контрольных сумм считывается, хеш-значения проверяемого файла вычисляются заново и сравниваются с эталонными по каждому алгоритму. Для каждого алгоритма выводится результат сравнения, а затем общий итог проверки. Код возврата программы позволяет использовать проверку в сценариях.

Для запуска программы предусмотрены следующие операции:

```
integrity hash examples/input.txt
integrity hash -s "строка"
integrity sign examples/input.txt examples/input.hash
integrity verify examples/input.txt examples/input.hash
integrity test
```

Первые две команды выводят хеш-значения файла или строки, третья формирует файл контрольных сумм, четвёртая проверяет целостность файла, а пятая выполняет проверку реализации на контрольных примерах. Для хеш-функции ГОСТ Р 34.11-2012 используются два примера из приложения стандарта: сообщение M_1 длиной 63 байта и сообщение M_2 длиной 72 байта, которое занимает два блока. Для SHA-1 используются примеры из стандарта FIPS 180, в том числе сообщение из миллиона символов «a». Результат проверки продемонстрирован на рисунке 1.1.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab5$ ./build/integrity test
  GOST-256 M1    OK
  GOST-512 M1    OK
  GOST-256 M2    OK
  GOST-512 M2    OK
  SHA-1 abc      OK
  SHA-1 empty    OK
  SHA-1 448 bit  OK
  SHA-1 1M 'a'   OK
  Все тесты пройдены
```

Рисунок 1.1 – Проверка реализации на контрольных примерах из стандартов

Все вычисленные значения совпали с приведёнными в стандартах, что подтверждает правильность реализации обоих алгоритмов.

Для проверки контроля целостности для тестового файла input.txt был сформирован файл контрольных сумм input.hash, что продемонстрировано на рисунке 1.2.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab5$ ./build/integrity sign examples/input
  .txt examples/input.hash
  GOST34.11-2012-256: e929e2a7805a7e52d5fdc410785d28db6250493082f38bfdcd555aaf8d653c17
  GOST34.11-2012-512: 8befb34d6c8b25e2e07dd6eba4caad56bf4c5ba8a1f41336fd3b8a35d297756b7e320e21f08a46f4f8ef
  5e3c7deb38ec83b6f56368f8c43fca44895ffec5c944
  SHA-1:              1b6a959ab33601327a4533021704be5218c346e0
  Сохранено: examples/input.hash

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab5$ cat examples/input.hash
  file: input.txt
  size: 1086
  GOST34.11-2012-256: e929e2a7805a7e52d5fdc410785d28db6250493082f38bfdcd555aaf8d653c17
  GOST34.11-2012-512: 8befb34d6c8b25e2e07dd6eba4caad56bf4c5ba8a1f41336fd3b8a35d297756b7e320e21f08a46f4f8ef
  5e3c7deb38ec83b6f56368f8c43fca44895ffec5c944
  SHA-1: 1b6a959ab33601327a4533021704be5218c346e0
```

Рисунок 1.2 – Формирование файла контрольных сумм

Повторная проверка неизменённого файла показывает совпадение всех трёх хеш-значений с эталонными, что продемонстрировано на рисунке 1.3.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab5$ ./build/integrity verify examples/inp
ut.txt examples/input.hash
GOST34.11-2012-256: OK
GOST34.11-2012-512: OK
SHA-1:              OK
Целостность подтверждена
```

Рисунок 1.3 – Проверка целостности неизменённого файла

Затем был создан файл modified.txt, в котором по сравнению с исходным изменён один символ: число 80 заменено на 81, то есть изменён всего один бит. Размер файла при этом не изменился. Как видно на рисунке 1.4, все три хеш-значения перестали совпадать с эталонными, и программа сообщила о нарушении целостности.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab5$ diff examples/input.txt examples/modi
fied.txt
9c9
< SHA-1 (FIPS 180-4): блок 512 бит, выход 160 бит, 80 раундов обработки.
---
> SHA-1 (FIPS 180-4): блок 512 бит, выход 160 бит, 81 раундов обработки.

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab5$ ./build/integrity verify examples/mod
ified.txt examples/input.hash
GOST34.11-2012-256: НЕ СОВПАДАЕТ
GOST34.11-2012-512: НЕ СОВПАДАЕТ
SHA-1:              НЕ СОВПАДАЕТ
Целостность нарушена
```

Рисунок 1.4 – Проверка целостности изменённого файла

Таким образом, в ходе работы были реализованы хеш-функции ГОСТ Р 34.11-2012 с длиной выхода 256 и 512 бит и SHA-1, а на их основе – программное средство контроля целостности файлов.

ВЫВОД

В ходе выполнения лабораторной работы были изучены криптографические хеш-функции и разработано программное средство на языке C++, выполняющее контроль целостности сообщений с помощью хеш-функций ГОСТ 34.11 и SHA-1.

В программе самостоятельно реализованы преобразования X, S, P и L, функция сжатия и полная процедура вычисления хеш-функции ГОСТ Р 34.11-2012 для длины выхода 256 и 512 бит, а также дополнение сообщения, расширение блока до 80 слов и 80 циклов обработки алгоритма SHA-1. Корректность реализации подтверждена совпадением результатов с контрольными примерами из ГОСТ Р 34.11-2012 и FIPS 180.

Проведённое тестирование показало, что неизменённый файл успешно проходит проверку по сохранённым контрольным суммам, а изменение даже одного бита приводит к несовпадению всех хеш-значений.

В результате выполнения лабораторной работы были получены практические навыки реализации криптографических хеш-функций, работы с битовыми и байтовыми преобразованиями, арифметикой по модулю 2^{32} и 2^{512}, а также организации контроля целостности файлов с помощью хеш-значений.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – streebog.h

```
#pragma once

#include <array>
#include <cstddef>
#include <cstdint>
#include <vector>

class Streebog {
public:
    using Block = std::array<uint8_t, 64>;

    explicit Streebog(int bits = 512);

    void update(const uint8_t* data, size_t len);
    void update(const std::vector<uint8_t>& data) {
        update(data.data(), data.size());
    }
    std::vector<uint8_t> digest();

    int bits() const { return bits_; }

    static std::vector<uint8_t> hash(const std::vector<uint8_t>& data,
                                     int bits);

private:
    void compress(const Block& m, size_t lenBits);

    int bits_;
    Block h_{}, N_{}, Sigma_{};
    Block buf_{};
    size_t bufLen_ = 0;
    bool done_ = false;
};
```

Листинг А.2 – sha1.h

```
#pragma once

#include <array>
#include <cstddef>
#include <cstdint>
#include <vector>

class Sha1 {
public:
    Sha1();

    void update(const uint8_t* data, size_t len);
    void update(const std::vector<uint8_t>& data) {
        update(data.data(), data.size());
    }
    std::vector<uint8_t> digest();

    static std::vector<uint8_t> hash(const std::vector<uint8_t>& data);

private:
```

```
    void processBlock(const uint8_t* block);

    std::array<uint32_t, 5> H_;
    std::array<uint8_t, 64> buf_{};
    size_t bufLen_ = 0;
    uint64_t totalBits_ = 0;
    bool done_ = false;
};
```

Листинг А.3 – sha1.cpp

```
#include "sha1.h"

#include <algorithm>
#include <stdexcept>

namespace {

uint32_t rotl(uint32_t x, int s) {
    return x << s | x >> (32 - s);
}

uint32_t f(int t, uint32_t b, uint32_t c, uint32_t d) {
    if (t < 20) return (b & c) | (~b & d);
    if (t < 40) return b ^ c ^ d;
    if (t < 60) return (b & c) | (b & d) | (c & d);
    return b ^ c ^ d;
}

uint32_t K(int t) {
    if (t < 20) return 0x5A827999;
    if (t < 40) return 0x6ED9EBA1;
    if (t < 60) return 0x8F1BBCDC;
    return 0xCA62C1D6;
}

}

Sha1::Sha1()
    : H_{0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476, 0xC3D2E1F0} {}

void Sha1::processBlock(const uint8_t* block) {
    uint32_t W[80];
    for (int t = 0; t < 16; ++t)
        W[t] = uint32_t(block[4 * t]) << 24 |
               uint32_t(block[4 * t + 1]) << 16 |
               uint32_t(block[4 * t + 2]) << 8 | uint32_t(block[4 * t + 3]);
    for (int t = 16; t < 80; ++t)
        W[t] = rotl(W[t - 3] ^ W[t - 8] ^ W[t - 14] ^ W[t - 16], 1);

    uint32_t a = H_[0], b = H_[1], c = H_[2], d = H_[3], e = H_[4];
    for (int t = 0; t < 80; ++t) {
        const uint32_t tmp = rotl(a, 5) + f(t, b, c, d) + e + W[t] + K(t);
        e = d;
        d = c;
        c = rotl(b, 30);
        b = a;
        a = tmp;
    }
    H_[0] += a;
    H_[1] += b;
    H_[2] += c;
```

```
    H_[3] += d;
    H_[4] += e;
}

void Sha1::update(const uint8_t* data, size_t len) {
    if (done_) throw std::logic_error("Sha1: хеш уже вычислен");
    totalBits_ += uint64_t(len) * 8;
    while (len > 0) {
        const size_t take = std::min(len, 64 - bufLen_);
        for (size_t i = 0; i < take; ++i) buf_[bufLen_ + i] = data[i];
        bufLen_ += take;
        data += take;
        len -= take;
        if (bufLen_ == 64) {
            processBlock(buf_.data());
            bufLen_ = 0;
        }
    }
}

std::vector<uint8_t> Sha1::digest() {
    if (done_) throw std::logic_error("Sha1: хеш уже вычислен");
    const uint64_t bits = totalBits_;

    const uint8_t one = 0x80, zero = 0x00;
    update(&one, 1);
    while (bufLen_ != 56) update(&zero, 1);

    uint8_t len[8];
    for (int i = 0; i < 8; ++i) len[i] = uint8_t(bits >> (56 - 8 * i));
    update(len, 8);
    done_ = true;

    std::vector<uint8_t> out(20);
    for (int i = 0; i < 5; ++i)
        for (int j = 0; j < 4; ++j)
            out[4 * i + j] = uint8_t(H_[i] >> (24 - 8 * j));
    return out;
}

std::vector<uint8_t> Sha1::hash(const std::vector<uint8_t>& data) {
    Sha1 s;
    s.update(data);
    return s.digest();
}
```